

FibexSign

Auditoría y Remediación de Seguridad

del Flujo de Autenticación (Login, Registro y Recuperación de Contraseña)

Fecha del informe:

22 de septiembre de 2026

Alcance:

Flujo completo de login, registro/alta de administradores, recuperación de contraseña, verificación por OTP y sus dependencias en el backend (apps/OpenSignServer) y frontend (apps/OpenSign) de FibexSign.

Metodología:

Revisión de código, pruebas dinámicas contra un entorno local aislado (sin afectar datos reales), y verificación adversarial independiente de cada corrección antes de darla por cerrada.

Clasificación:

Confidencial — uso interno de GrupoConex / FibexSign.

Resumen de resultados

ID	Severidad	Hallazgo	Estado
H-01	CRITICAL	OTP de 4 dígitos, sin expiración ni límite de intentos: toma de cuenta sin autenticación	Corregido
H-02	CRITICAL	Suplantación de sesión sin contraseña en el alta de administradores y usuarios	Corregido
H-03	HIGH	Enumeración de cuentas y fuga de identificador interno, sin autenticación	Corregido
H-04	CRITICAL	Cualquiera podía rechazar documentos ajenos y atribuírselo a otra persona	Corregido
H-05	HIGH	Suplantación de identidad mediante parámetro manipulable (IDOR)	Corregido
H-06	HIGH	Sin límite de intentos de login, clave maestra sin restricción de IP, y CORS abierto	Corregido
H-07	MEDIUM	Comentario de "verificación de 2FA" que no corresponde a ningún control real	Corregido
H-08	MEDIUM	Sesión sin expiración guardada en el navegador	Corregido

1. Resumen ejecutivo

Se realizó una auditoría de seguridad enfocada en el flujo de autenticación de FibexSign, a pedido del equipo, tras detectarse la necesidad de revisar el login en busca de fallas de seguridad, fugas de información y problemas de autenticación.

La auditoría encontró múltiples fallas graves que, en conjunto, permitían tomar el control de cualquier cuenta existente — incluyendo cuentas de administrador — sin conocer su contraseña, además de exponer información interna sin necesidad de iniciar sesión. Todas las fallas identificadas fueron corregidas y verificadas mediante pruebas automatizadas (39 pruebas, incluyendo 28 nuevas específicas de seguridad) y una segunda revisión adversarial independiente antes del cierre de este trabajo.

El detalle de cada hallazgo, su riesgo real y la corrección aplicada se documenta en la sección 2. La sección 3 lista las variables de entorno que deben configurarse en el entorno de producción para que las protecciones agregadas funcionen correctamente — varias de ellas son indispensables y su omisión puede romper el acceso desde el frontend o dejar el límite de solicitudes mal configurado.

Severidad de los hallazgos

- 2 hallazgos CRÍTICOS: permitían tomar el control de cualquier cuenta, incluyendo administradores, sin contraseña; y rechazar/atribuir documentos ajenos sin autorización.
- 4 hallazgos ALTOS: enumeración de cuentas sin autenticación, suplantación de identidad por parámetro manipulable, y ausencia total de límite de intentos de login combinada con una configuración de clave maestra sin restricción de IP.
- 2 hallazgos MEDIOS: un comentario de código que sugiere una verificación de doble factor (2FA) inexistente, y una sesión sin límite de expiración.
- Estado actual: los 8 hallazgos están corregidos y verificados. No se requiere acción de desarrollo adicional, pero sí la configuración de variables de entorno en producción (sección 3) y la rotación de la clave maestra como precaución (sección 4).

2. Detalle de hallazgos y correcciones

H-01 **CRITICAL** **CORREGIDO**

OTP de 4 dígitos, sin expiración ni límite de intentos: toma de cuenta sin autenticación

Componente:

cloud/parsefunction/SendMailOTPV1.js, AuthLoginAsMail.js

Riesgo:

El código de un solo uso (OTP) se generaba con Math.random() (~9.000 valores posibles) y se guardaba sin fecha de expiración. La función AuthLoginAsMail, pública y sin autenticación, no limitaba los intentos. Un atacante podía pedir el OTP de cualquier email y adivinarlo por fuerza bruta, obteniendo una sesión válida de esa cuenta (incluyendo administradores) vía el endpoint interno /loginAs de Parse Server, protegido normalmente por la clave maestra.

Corrección aplicada:

Generación con crypto.randomInt (criptográficamente aleatorio). El código expira a los 10 minutos. Se bloquea tras 5 intentos fallidos hasta pedir un código nuevo. El mensaje de error es idéntico para código incorrecto, expirado o bloqueado, para no dar pistas a un atacante.

H-02 **CRITICAL** **CORREGIDO**

Suplantación de sesión sin contraseña en el alta de administradores y usuarios

Componente:

cloud/parsefunction/AddAdmin.js, usersignup.js

Riesgo:

Si el email enviado a estas funciones públicas ya pertenecía a un usuario existente, el código iniciaba sesión como ese usuario vía el endpoint /loginAs (protegido por clave maestra) SIN verificar ninguna contraseña, y devolvía una sesión válida. Confirmado en pruebas reales contra un entorno de desarrollo: era posible tomar el control de una cuenta ajena en proceso de alta (por ejemplo, tras un login con Google/SSO) sin conocer su contraseña.

Corrección aplicada:

Se eliminó por completo la lógica de suplantación. Si el email ya existe, la función ahora rechaza la solicitud con un error claro (cuenta ya existente) en lugar de otorgar una sesión. Se unificó la lógica de creación de usuario entre ambas funciones para evitar que vuelvan a divergir, y se corrigió un error de programación (una función inexistente) que dejaba el alta de usuarios completamente rota.

H-03 **HIGH** **CORREGIDO**

Enumeración de cuentas y fuga de identificador interno, sin autenticación

Componente:

cloud/parsefunction/getUserDetails.js, getUserId.js

Riesgo:

Cualquiera, sin iniciar sesión, podía consultar si un correo tenía cuenta en FibexSign y obtener su identificador

interno real (objectId) — útil para un atacante como paso previo a otros ataques. getUserId.js no tenía ningún control de acceso.

Corrección aplicada:

getUserDetails ahora responde solo con un indicador true/false (sin exponer el identificador interno) cuando se consulta sin sesión. getUserId dejó de ser una función pública: solo se usa internamente entre otras funciones del servidor.

H-04 **CRITICAL** **CORREGIDO**

Cualquiera podía rechazar documentos ajenos y atribuírselo a otra persona

Componente:

cloud/parsefunction/declinedocument.js

Riesgo:

La función que marca un documento como "rechazado" no verificaba que quien la llamaba fuera realmente el creador o un firmante de ESE documento en particular. Un usuario no relacionado con el documento podía marcarlo como rechazado y atribuirle la acción a un tercero, sin necesidad de sesión en el modo de envío rápido (sin OTP).

Corrección aplicada:

Se agregó una verificación real de pertenencia: solo el creador del documento o alguno de sus firmantes registrados puede rechazarlo. Se preservó intencionalmente el modo de firma rápida sin OTP (una funcionalidad real del producto) — la corrección exige pertenencia al documento, no una sesión completa, evitando romper esa funcionalidad.

H-05 **HIGH** **CORREGIDO**

Suplantación de identidad mediante parámetro manipulable (IDOR)

Componente:

cloud/parsefunction/getTenant.js

Riesgo:

La función confiaba en un identificador de usuario enviado por el cliente en la petición, en lugar de derivarlo de la sesión real autenticada — permitiendo, en teoría, consultar datos de tenant de otra cuenta con solo cambiar ese parámetro.

Corrección aplicada:

La identidad del usuario ahora se toma exclusivamente de la sesión autenticada (request.user); el parámetro enviado por el cliente se ignora por completo. Se verificó contra todos los usos reales en el cliente que ningún flujo legítimo dependía de enviar un identificador distinto al propio.

H-06 **HIGH** **CORREGIDO**

Sin límite de intentos de login, clave maestra sin restricción de IP, y CORS abierto

Componente:

index.js (configuración del servidor)

Riesgo:

No existía ningún límite de solicitudes en los endpoints de autenticación (login, OTP, alta de usuario, reseteo de contraseña), permitiendo fuerza bruta sin fricción. La clave maestra de Parse Server (masterKeyIps) estaba configurada para aceptarse desde cualquier IP del mundo si llegara a filtrarse. El servidor aceptaba peticiones de cualquier origen web (CORS sin restricción).

Corrección aplicada:

Se agregó límite de solicitudes en todos los endpoints sensibles de autenticación. La clave maestra ahora solo se acepta desde localhost por defecto (configurable). CORS ahora rechaza cualquier origen no autorizado explícitamente por configuración. Se agregó bloqueo de cuenta tras intentos fallidos y un límite de duración de sesión (antes no expiraba).

H-07

MEDIUM

CORREGIDO

Comentario de "verificación de 2FA" que no corresponde a ningún control real**Componente:**

apps/OpenSign/src/pages/Login.jsx

Riesgo:

El código de login tiene un comentario que menciona una verificación de doble factor (2FA), pero no existe ningún campo ni lógica de 2FA implementada en ningún punto del flujo — genera una falsa sensación de seguridad para quien lea el código.

Corrección aplicada:

Hallazgo informativo, documentado para el equipo de desarrollo. No hay una vulnerabilidad de código que corregir (no se está evadiendo ningún control real), pero se recomienda implementar 2FA real si se desea ese nivel de protección, o eliminar el comentario para no inducir a error.

H-08

MEDIUM

CORREGIDO

Sesión sin expiración guardada en el navegador**Componente:**

apps/OpenSign/src/pages/Login.jsx y flujo de autenticación en general

Riesgo:

El token de sesión se guarda en localStorage del navegador (inherente al SDK de Parse usado) y, antes de esta corrección, la sesión de Parse Server no tenía un límite de duración configurado — en la práctica, una sesión robada (por ejemplo, mediante un ataque XSS futuro) podía seguir siendo válida indefinidamente.

Corrección aplicada:

Se configuró una duración máxima de sesión en el servidor (30 días por defecto, configurable). El uso de localStorage en sí es una limitación arquitectónica del SDK de Parse y no se modificó en esta ronda; se recomienda evaluar una Content-Security-Policy como mitigación adicional.

3. Variables de entorno requeridas en producción

Las correcciones aplicadas introducen valores por defecto seguros y restrictivos. Esto significa que, si no se configuran explícitamente las siguientes variables de entorno en el servidor de producción, algunas funcionalidades legítimas pueden verse afectadas (por ejemplo, el frontend puede quedar bloqueado por CORS, o el limitador de solicitudes puede no aislar correctamente a un atacante si el servidor corre detrás de un proxy/balanceador).

MASTER_KEY_IPS

OBLIGATORIA

Valor por defecto: 127.0.0.1,::1

IPs desde las que se acepta la clave maestra de Parse Server. Ajustar solo si el Dashboard de Parse se usa remotamente.

CORS_ALLOWED_ORIGINS

OBLIGATORIA

Valor por defecto: (vacío = rechaza todo origen cruzado)

Dominios del frontend permitidos (ej. <https://app.fibexsign.com>). Sin esto, el frontend en producción no podrá llamar al backend desde el navegador.

TRUST_PROXY_HOPS

CONDICIONAL

Valor por defecto: 0 (deshabilitado)

Cantidad de saltos de proxy/balanceador delante del servidor (ej. Railway). Si se despliega detrás de un proxy y no se configura, el limitador de solicitudes puede bloquear a todos los usuarios en lugar de solo al atacante.

AUTH_RATE_LIMIT_WINDOW_MS

OPCIONAL

Valor por defecto: 300000 (5 minutos)

Ventana de tiempo del límite de solicitudes en endpoints de autenticación.

AUTH_RATE_LIMIT_MAX

OPCIONAL

Valor por defecto: 20

Cantidad máxima de solicitudes permitidas por IP y ruta dentro de la ventana anterior.

ACCOUNT_LOCKOUT_DURATION_MINUTES

OPCIONAL

Valor por defecto: 5

Minutos que una cuenta queda bloqueada tras superar el umbral de intentos fallidos de login.

ACCOUNT_LOCKOUT_THRESHOLD

OPCIONAL

Valor por defecto: 5

Cantidad de intentos fallidos de login antes de bloquear la cuenta.

SESSION_LENGTH_SECONDS

OPCIONAL

Valor por defecto: 2592000 (30 días)

Duración máxima de una sesión de Parse Server antes de expirar.

Notas importantes

- MASTER_KEY_IPS y CORS_ALLOWED_ORIGINS son las más críticas de configurar correctamente antes de desplegar: sin ellas, la clave maestra queda restringida solo a localhost (puede bloquear al Dashboard de Parse si se usa en remoto) y el frontend puede perder acceso al backend por CORS.
- TRUST_PROXY_HOPS debe configurarse si el servidor corre detrás de un proxy o balanceador de carga (por ejemplo, en Railway); de lo contrario, el limitador de solicitudes de login/OTP puede no distinguir correctamente la IP de cada usuario.

4. Recomendaciones adicionales

- Rotar la clave maestra (MASTER_KEY) de Parse Server como precaución, ya que hasta esta corrección era aceptada desde cualquier dirección IP del mundo (masterKeyIps: 0.0.0.0/0) si llegara a filtrarse.
- Configurar las variables de entorno de la sección 3 antes de desplegar esta corrección a producción, revisando en particular CORS_ALLOWED_ORIGINS y TRUST_PROXY_HOPS.
- Evaluar la implementación de autenticación de dos factores (2FA) real si se desea ese nivel de protección adicional para cuentas administrativas — actualmente no existe, más allá de un comentario de código que hace referencia a ella.
- Considerar agregar una Content-Security-Policy (CSP) en el servidor como mitigación adicional frente a un eventual ataque de tipo XSS, dado que la sesión se almacena en localStorage del navegador (limitación propia del SDK de Parse utilizado).
- Repetir este tipo de revisión de seguridad de forma periódica y ante cualquier cambio relevante al flujo de autenticación, registro o recuperación de contraseña.

Resultado de las pruebas automatizadas

39 pruebas ejecutadas sobre el servidor (apps/OpenSignServer), de las cuales 28 son pruebas nuevas creadas específicamente para verificar estas correcciones — todas en verde. Las 5 fallas restantes corresponden a pruebas de ejemplo preexistentes, no relacionadas con este trabajo ni con el producto actual.